Assignment 5: Database Normalization and ER Diagram Analysis

Analyze the Project Requirements

Identify some of the main entities

For Team 1 Cybersecurity project, I would create a data base that helps a financial institution that tracks incidents, vulnerabilities, affected systems, and the employees responsible for resolving the security issues.

With the requirements needed for the projects, I would propose these following tables:

Employees

This table would store information about the employees who handle the security incidents and vulnerabilities

- EmployeeID
- FirstName
- LastName
- JobTitle
- Email
-

Assets

This table would store information about the company's hardware such as computers, servers, and applications

- AssetID
- AssetName
- AssetType
- OperatingSystem
- AssetOwnerID

Security Incidents

This table will store information about reported incidents

- IncidentID
- IncidentDescription
- Severity
- DateReported
- Status
- AssignedEmployeeID

Vulnerabilities

This table will store information about vulnerabilities discovered during scans and audits.

- VulnerabilityID
- VulnerabilityName
- Description
- Severity
- DateDiscovered
- Status

Remediation Records

This table will track actions taken to fix the security issues

- RemediationID
- VulnerabilityID
- AssignedEmployeeID
- ActionTaken
- DateCompleted
- VerificationStatus

AssetVulernabilities

This table will track which vulnerabilities affect specific company assets

- AssetID
- VulnerabilityID
- DateDetected
- RemediationStatus

Why should the information be organized into separate tables?

The information should be in separate tables because each table will represent a different type of information. For example, an employee might be responsible for multiple security incidents, so if I were to store everything in one large table, I would have to repeat the employee's information for every incident that is assigned to that particular employee and it would create unnecessary duplicate information and make the database harder to maintain.

Separating the employees and incidents into different tables could make it easier by storing the employee's information once and use the EmployeeID to identify which employee is responsible for each incident.

If there would be any changes with the employee's details, only one record would be updated instead of multiple incidents and with this approach it would make the database more organized, reduce duplicate information, and help prevent inconsistent data.

Apply Database Normalization

First Normal Form (1NF)

First Normal Form means that each column should contain a single value rather than multiple values combined into one field. Each record would also have an unique identifier.

Example, suppose a security incident affects three different servers, I would not store all three server names in one column as it would be three separate incidents in a row using the same IncidentID

IncidentID	AssetID
101	1
101	2
101	3

Each row would represent one incident and the combination of the IncidentID and AssetID would serve as a composite primary key.

Second Normal Form (2NF)

A 2NF would mean taht the database must already follow the 1NF and that every no key column must depend on the primary key.

Example, let's say there is an AssetVulnerabilities table that uses AssetID and VulnerabilityID together to idenftiy which vulnerabilities affect specific assets like the table underneath,

AssetID	VulnerabilityID	AssetName	DateDetected
1	100	WebServer01	2026-10-01
1	101	WebServer01	2026-10-05

This would cause a problem with the AssetName as it would depend on AssietID and not both AssetID and VaulnerabilityID. For 2NF to work it would be something like this,

Assets

AssetID	AssetName
1	WebServer01

AssetVulnerabilities

AssetID	VulnerabilityID	DateDetected
1	100	2026-10-01
1	101	2026-10-05

With this method, assets are stored separately while DataDetected describes the specific combination while preventing unnecessary duplication of asset information.

Third Normal Form (3NF)

A 3NF would mean that the database must follow the 2NF rules and that columns with no keys should not depend on other no key columns.

For example, I have a SecurityIncidents table with the following information underneath,

IncidentID	AssignedEmployeeID	EmployeeName	EmployeeEmail
101	5	John Smith	john@example.com
102	5	John Smith	john@example.com

This would be a problem because EmployeeName and EmployeeEmail depends on AssignedEmployeeID rather than the IncidentID, so if the employee changes anythign about their information, there would be multiple updates for the incident records.

The 3NF rules would follow something like this,

Employees

EmployeeID	EmployeeName	EmployeeEmail
5	John Smith	john@example.com

SecurityIncidents

IncidentID	AssignedEmployeeID	Severity
101	5	High
102	5	Medium

With this way, employee information is all stored under Employees table and the SecurityIncidents table only reference from the employee through AssignedEmployeeID.

Identify Relationships Between Tables

Relationship 1: Employees and SecurityIncidents

Relationship type: One to many

Employees and SecurityIncidents would be one to many because an employee can be assigned to multiple incidents but each incident would have one primary employee responsible for handling it.

So the Employees table would use EmployeeID as its primary key and the SecurityIncidents table would contain an AssignedEmployeeID as a foreign key. This would allow the database to identify the employee responsible for each incident without repeating their personal information.

Relationship 2: Assets and Vulnerabilities

Relationship type: Many to many

Assets and Vulnerabilities would be many to many because one vulnerability can affect multiple assets and one asset can have multiple vulnerabilities.

Example, a security vulnerability affecting a particular software version might be discovered on 10 different company servers and at the same time, one server might have several vulernabilities.

To represent this relationship, I would create an associative table called AssetVulnerabilities and The table would contain AssetID and VulnerabilityID as foreign keys referencing their respective tables

Relationship 3: Employees and RemediationRecords

Relationship type: One to many

Employees and RemediationRecords would be one to many because one IT administrator may perform multiple remediation actions, but each remediation record would identify the employee responsible for that specific action. The RemediationRecords table would store the employee's ID, the action performed, and the completion date and This would help the organization track which employees performed remediation work and maintain records for security audits.

Consider Real World Queries

Question 1: Which company assets currently have high-severity vulnerabilities that have not been resolved?

I would use the following tables:

- Assets - identify the affected computers, servers, or applications
- Vulnerabilities - provide information about each vulnerability, including its severity
- AssetVulnerabilities - help security analysts prioritize which systems need immediate attention

Question 2: Which employees are responsible for unresolved security incidents, and how severe are those incidents?

I would use:

- Employees - provide information about the employees responsible for investigating security incidents
- SecurityIncidents - contain the incident descriptions, severity levels, assigned employees, and current statuses

Those tables would be connected using EmployeeID and AssignedEmployeeID as it would allow easier identification when an incident is unresolved, determine the severity, and/or to see who is responsible for investigation. It would also help the organization manage the employee's workload and prioritize important security incidents.

Conclusion

For our cybersecurity database project, I believe separating information into tables such as Employees, Assets, SecurityIncidents, Vulnerabilities, and RemediationRecords would provide a good starting point.

Applying normalization would help reduce duplicate information, maintain consistency, and make the database easier to update.

Using primary keys and foreign keys would allow the tables to connect while keeping different types of information organized.

Although this is only my initial proposed design, I think it would provide a foundation for tracking security incidents, managing vulnerabilities, and helping the organization respond to cybersecurity threats more efficiently.